

Beyond univariate analysis

Predicting species richness with model-based ordination

Bert van der Veen

A short outline of gllvm

A Generalised Linear Latent Variable Model (GLLVM) is kind of like a multispecies GLM with a few extra terms. Each species j gets its own intercept and its own responses to the covariates, exactly as it would in a stack of separate GLMs, and on top of that every site i gets a small number of latent variables $\mathbf{u}_i$:

$$g\{E(y_{ij}|\mathbf{u}_i)\} = \beta_{0j} + \mathbf{x}_i^\top \boldsymbol{\beta}_j + \mathbf{u}_i^\top \boldsymbol{\gamma}_j$$

The latent variables are the part that makes this more than a loop over species. They are unobserved gradients, estimated from the data, that absorb the correlation left between species after the measured covariates have done their work, so species are no longer treated as independent. Because there are only a couple of them, and each species is represented by a loading vector $\boldsymbol{\gamma}_j$, this is also an ordination: $\mathbf{u}_i$ are site scores and $\boldsymbol{\gamma}_j$ species loadings, but fit by maximum likelihood under a response distribution chosen to match the data. This means we can do the same as classical ordination methods (PCA, RDA, CCA and so on, but better), but also a lot more (like Joint Species Distribution Modeling).

The `gllvm` package fits these models with `gllvm()`, using TMB for automatic differentiation and variational or Laplace approximations for any random effects. A few things matter for this practical:

- `family` picks the response distribution ("ordinal" here, but also "poisson", "negative.binomial", "binomial", "tweedie", "orderedBeta", and others), so the model is fit on the scale the data were recorded on.
- `num.lv` sets the number of *unconstrained* latent variables, estimated freely per site (residual ordination).
- `num.RR` sets the number of *constrained* axes, which are linear combinations of covariates given in `lv.formula` (constrained ordination, the model-based counterpart of RDA/CCA).
- Everything is a fitted statistical model, so the usual machinery applies: `coefplot()`, `ordiplot()`, `predict()`, `goodnessOfFit()`, information criteria, and `predictSR()` for the richness predictions this practical is built around.

We will build up from a basic model with elevation and two unconstrained latent variables (JSDM), add a hump shape, then swap the free latent variables for a constrained ordination that can also carry a year effect, and finally compare the richness predictions against a naive Poisson GLM. Note, the `gllvm` R-package can do a lot more than shown here, for which I refer to the vignettes and my Physalia workshop repository.

Data

The Skabholmen dataset records field-layer vegetation on a five-degree Hult-Sernander-Du Rietz cover scale, so `Y` contains the values 0 to 5, with 0 meaning absent. It holds 68 species in 126 one-square-metre plot records on the island Skabholmen (Stockholm archipelago, Sweden). Those 126 records come from 64 plots spread over four transects, surveyed at the start and end of a six-year period (1978 and 1984); two plots were surveyed in only one of the two years, which is why there are 126 rows rather than 128. Elevation relative to the shoreline was measured once, in centimetres during the 1978 survey, as a proxy for time since

land emergence, since the island sits on a rising, post-glacial land-uplift coast. The data were collected by Cramer & Hytteborn (1987), who separated longer-term successional change, associated with elevation, from shorter-term fluctuation between the two survey years. The same data are a CANOCO example dataset, analysed with canonical correspondence analysis by ter Braak (1987).

Cramer, W. & Hytteborn, H. (1987). The separation of fluctuation and long-term change in vegetation dynamics of a rising seashore. *Vegetatio*, 69, 157–167.

ter Braak, C. J. F. (1987). The analysis of vegetation-environment relationships by canonical correspondence analysis. *Vegetatio*, 69, 69–77.

```
library(gllvm)

data(Skabbholmen, package = "gllvm")
Y <- Skabbholmen$Y
X <- Skabbholmen$X

X$Elevation.s <- c(scale(X$Elevation))
X$Year.s <- X$Year - min(X$Year)
Y <- Y[, colSums(Y > 0) > 4]           # keep species present in at least 5 of the 126
  ↪ records

idx <- order(colSums(Y > 0))[rep(mean(1:ncol(Y)), 2) + c(0, 1)]
Y <- Y[, c(idx, (1:ncol(Y))[-idx])]  # put the two median-frequency species first

invisible(TMB::openmp(parallel::detectCores() - 1, autopar = TRUE, DLL = "gllvm"))
```

Practical note: the species filter and reordering are conveniences, not requirements. This filter leaves 38 of the 68 species. A stricter filter is more stable to fit but throws away part of what “richness” means for the rest of the analysis. Ultimately, infrequently occurring or observed species do not add much to (average) richness. Keeping rare species in may present as abrupt changes in species richness predictions along gradients, i.e., the predictions will not look smooth. The reordering matters because the loadings of the first `num.lv` species are constrained for identifiability, so those species should be ones with enough observations to pin the axes down.

A basic multispecies model

We start with elevation as the only covariate, and read the species-specific elevation effects off a caterpillar plot (Figure 1).

```
model <- gllvm(Y, X, formula = ~Elevation.s, num.lv = 2, family = "ordinal", seed = 123)
coefplot(model)
```

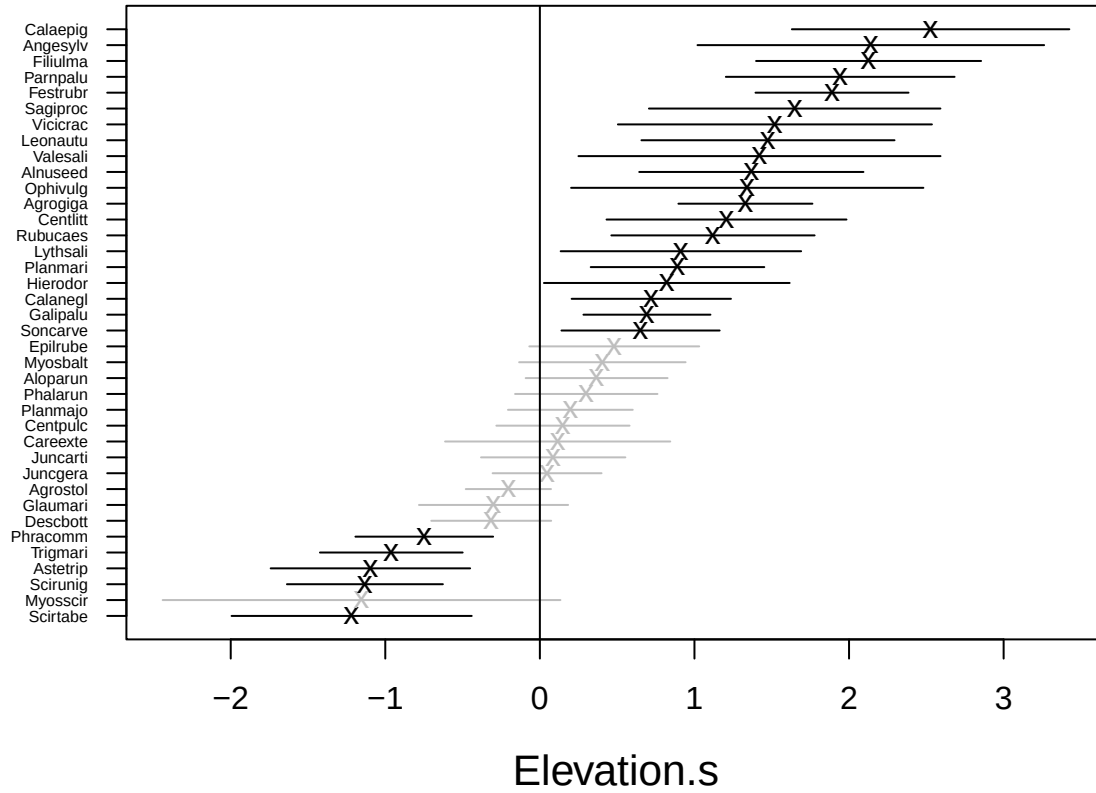


Figure 1: Species-specific elevation effects from the basic multispecies model, with confidence intervals.

`family = "ordinal"` fits a multinomial distribution to the data with cumulative probit link (alternatives are the logit link, and `cloglog` in the development version on github). Some species are never recorded in some cover classes, so `gllvm` cannot estimate species-specific cutoffs here and uses cutoffs common to all species instead (`zeta.struc = "common"`); it says so in a warning, which this document hides. `num.lv = 2` (default) adds latent variables that incorporate leftover correlation between species after elevation is accounted for. It is what makes this model a Joint Species Distribution Model.

Before reading anything into the species responses, check that the model fits the data, using its residual diagnostics.

```
par(mfrow = c(2, 3))
plot(model)
par(mfrow = c(1, 1))
```

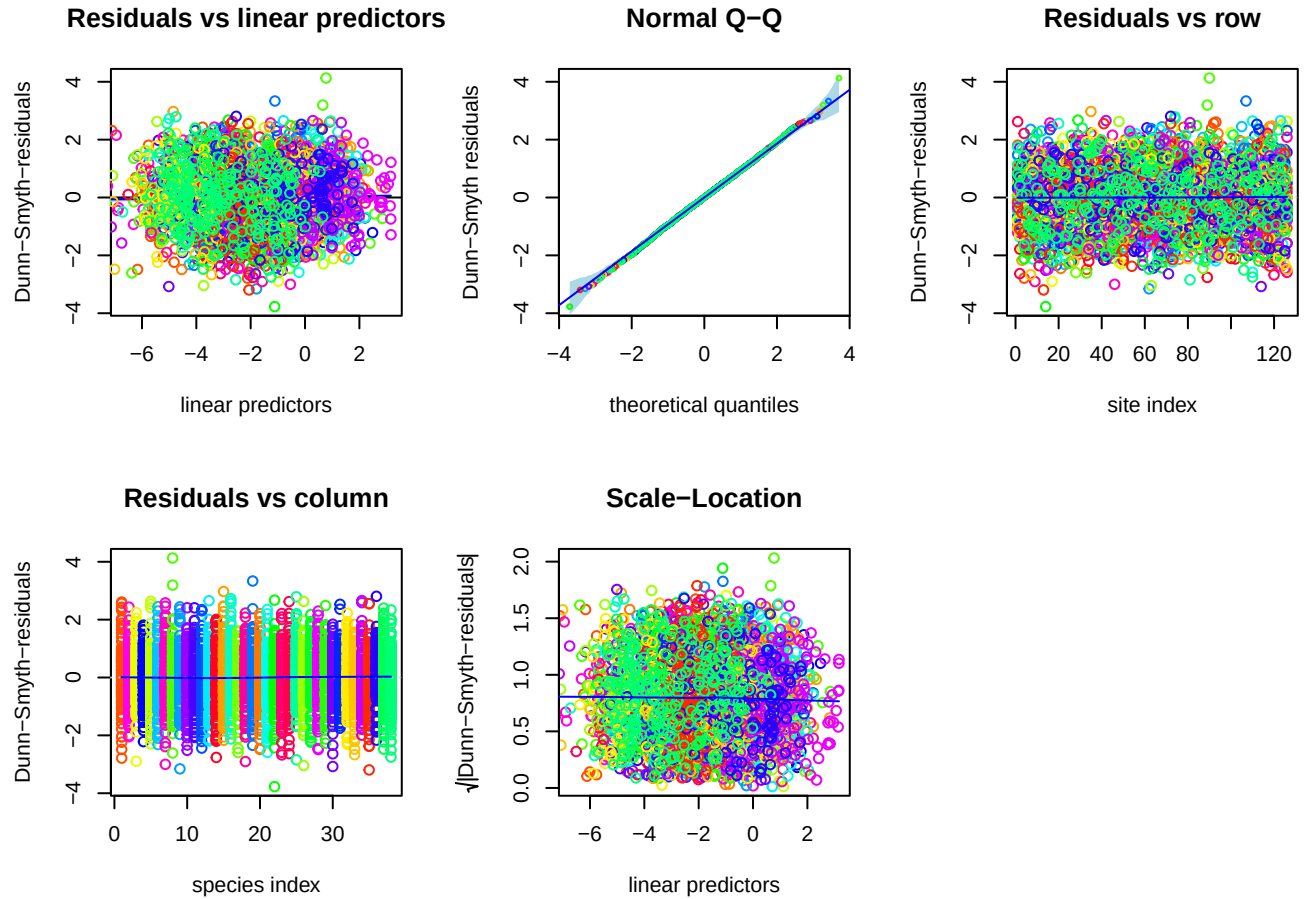


Figure 2: Dunn-Smyth residual diagnostics for the basic multispecies model: residuals against linear predictors, normal Q-Q plot, residuals against sites (rows) and species (columns), and scale-location plot.

The residuals are Dunn-Smyth residuals, so for a model that fits they should look standard normal: no pattern against the linear predictors, sites or species, and points along the line in the Q-Q plot.

The caterpillar plot in Figure 1 is the whole point of fitting the model this way: every species gets its own elevation slope, with its own uncertainty, and those slopes clearly do not agree. Some species increase with elevation, some decrease, some are flat, and the intervals differ in width because species differ in how often they were observed. Whether a species is present at a site is a function of exactly these species-specific responses, and richness is nothing more than the number of species present. So the richness response is built from these species-specific responses, disagreement included, which is exactly what a univariate richness model assumes away by treating all species as responding identically. That is the step we take next.

Predicting species richness

Why not just fit richness directly?

Richness is a sum of presence indicators, $SR_i = \sum_{j=1}^p \mathbb{I}(y_{ij} > 0)$, and the probability of a presence depends on the response distribution: for binary data it is π_{ij} directly, for Poisson counts it is $1 - e^{-\mu_{ij}}$, for ordinal cover classes it is one minus the probability of the zero class, for biomass or percentage cover it is one minus the point mass at zero under whatever distribution fits that response (Tweedie, ordered beta). Either way,

SR_i is a sum of non-identical, bounded indicators, which follows a Poisson-Binomial distribution, not a Poisson or a negative binomial, although those are what richness is usually handed to.

We do not model richness directly, because that would assume all species respond identically to the environment and are independent of each other: the assumptions outlined in the presentation, which do not hold for real communities. Fitting richness directly throws away everything the caterpillar plot just showed: the species-specific responses are collapsed into a single number before the model ever sees them. A multispecies model instead returns each species' presence probability under the response distribution the data were actually fitted with, and lets richness fall out as a derived prediction, correlation and natural boundedness included. The `gllvm` package computes the probability of presence with the function `gllvm::gllvm.presence.prob`, which is used under the hood by `gllvm::predictSR`, the function we will use in the practical for predicting richness.

Richness from the basic model

```
SR <- predictSR(model)
plot(SR)
```

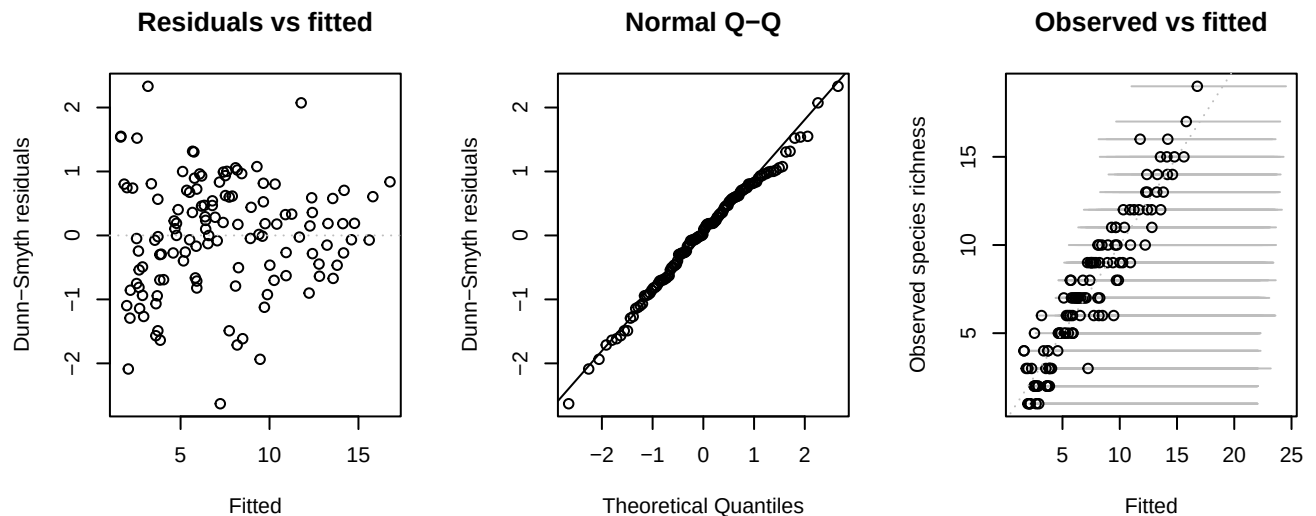


Figure 3: Richness diagnostics for the basic multispecies model, for the sites the model was fitted to: Dunn-Smyth residuals against fitted richness, their normal Q-Q plot, and observed against fitted richness with the simulated intervals as grey segments.

```
newELEV.s <- seq(min(X$Elevation.s), max(X$Elevation.s), length.out = 100)
newx <- newELEV.s * sd(X$Elevation) + mean(X$Elevation)
newLV <- matrix(0, ncol = 2, nrow = 100)

SRpred <- predictSR(model, newX = data.frame(Elevation.s = newELEV.s), newLV = newLV)

plot(y = rowSums(Y > 0), X$Elevation, xlab = "Elevation", ylab = "Species richness", col
  ↪ = "gray", pch = 20)
lines(y = SRpred$expected$fit, x = newx, col = "red")
lines(y = SRpred$expected$lower, x = newx, col = "red", lty = "dashed")
lines(y = SRpred$expected$upper, x = newx, col = "red", lty = "dashed")
```

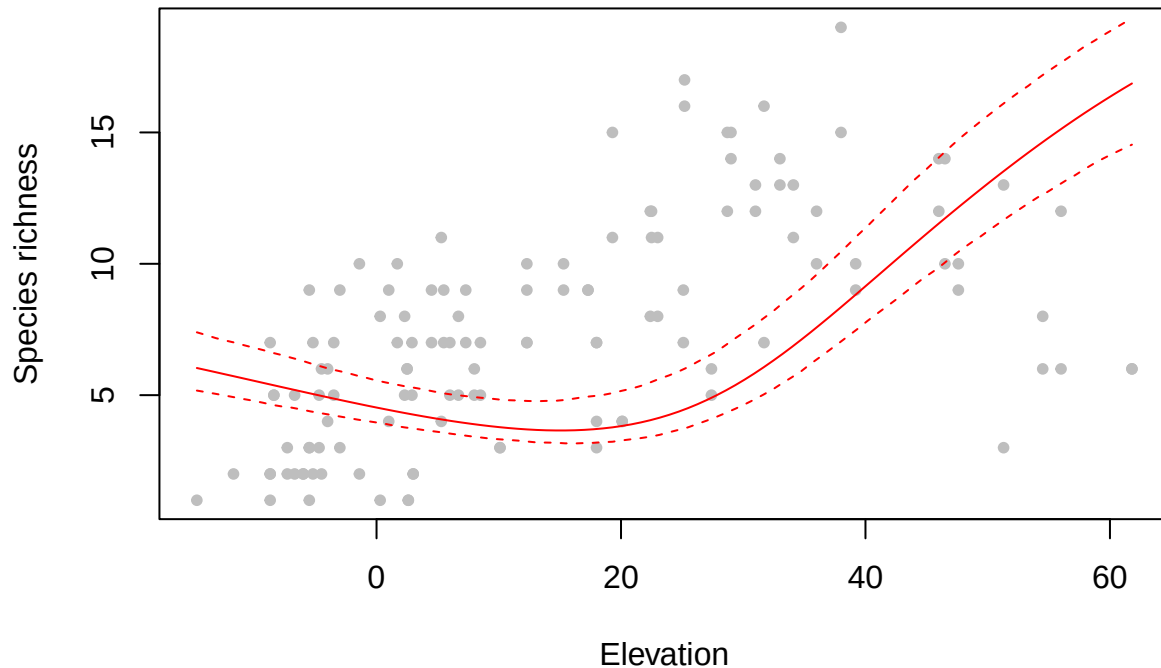


Figure 4: Predicted species richness along elevation from the basic multispecies model. Gray points are observed richness per site; the solid red line is the predicted mean richness; dashed red lines are the 95% confidence interval for that mean.

The red line is the model's predicted mean richness across the elevation gradient (latent variables held at zero), and the two dashed red lines around it are the boundaries of its 95% confidence interval, not a raw data range. The gray points are the actual observed richness values, plotted only to judge the fit by eye.

Practical note: setting `newLV` to zero gives the prediction at the mean of the latent variables, a common approximation for a partial-effect plot, not an exact marginal prediction (which would average the prediction over the distribution of the latent variables instead).

Adding a hump shape

```
model2 <- gllvm(Y, X, formula = ~Elevation.s + I(Elevation.s^2), num.lv = 2,
               family = "ordinal", seed = 8004)

SRpred2 <- predictSR(model2, newX = data.frame(Elevation.s = newELEV.s), newLV = newLV)

plot(y = rowSums(Y > 0), X$Elevation, xlab = "Elevation", ylab = "Species richness", col
     ↪ = "gray", pch = 20)
lines(y = SRpred2$expected$fit, x = newx, col = "red")
lines(y = SRpred2$expected$lower, x = newx, col = "red", lty = "dashed")
lines(y = SRpred2$expected$upper, x = newx, col = "red", lty = "dashed")
```

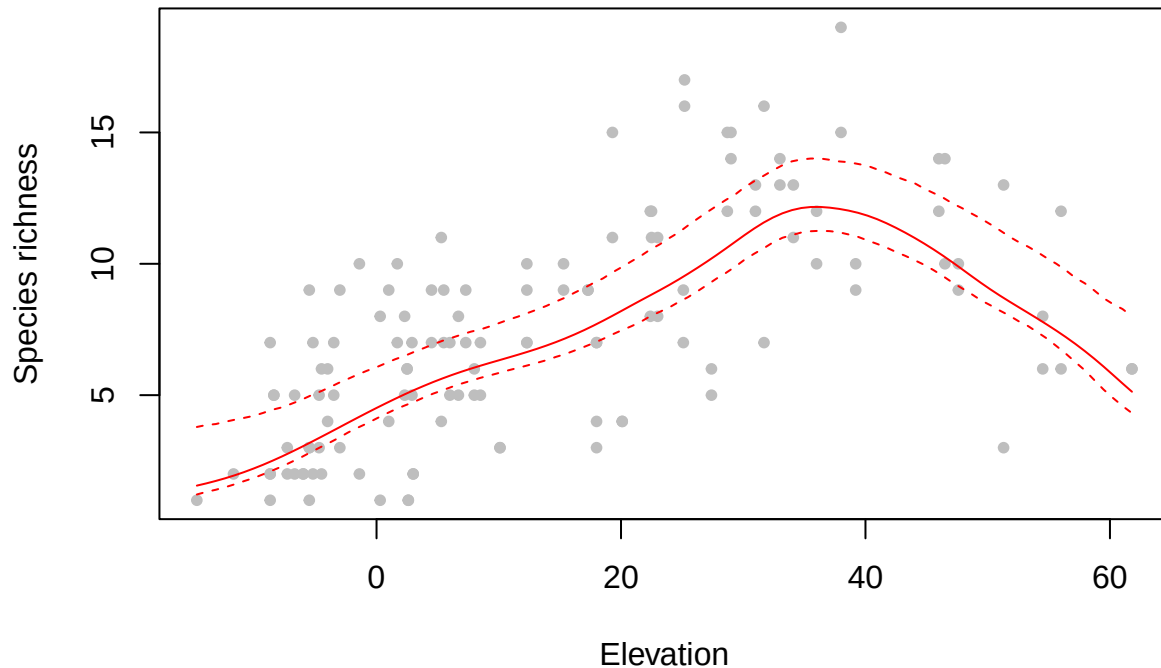


Figure 5: Predicted species richness along elevation after adding a quadratic elevation term (hump-shaped response).

Practical note: the likelihood is not guaranteed to be unimodal. If results need to be reproducible, fit with a few different seeds (or use `n.init`) and check that the fits agree before settling on one.

```
# Conditional on the estimated unconstrained latent variables
goodnessOfFit(model2, measure = "AUC")

# Fair comparison: latent variables fixed at zero, mimicking prediction for a new, unseen
↪ site
goodnessOfFit(model2, pred = predict(model2, newLV = matrix(0, ncol = 2, nrow = nrow(Y)),
                                     ordinal.cat = 1, type = "response"), measure =
↪ "AUC")
```

This chunk is not evaluated when the document is knitted, so run it yourself to see the two AUCs. The first AUC is conditional on the estimated unconstrained latent variables, which represent unobserved gradients or residual co-occurrence: information that is not available at new sites. The LV-zeroed number is the one that reflects prediction for new sites. Any gap between the two comes from the free, unconstrained latent variables, not from a weak elevation-richness relationship, and it is the argument for `model3` below: constrain the latent structure instead of letting every site have its own free score.

Adding a year effect: constrained ordination

A full interaction model ($\sim(\text{Elevation.s} + \text{I}(\text{Elevation.s}^2)) * \text{Year.s}$ per species) needs too many parameters for species with only a handful of observations. Constrained ordination fixes this by projecting the covariates onto a couple of shared axes instead of giving each species its own coefficient for everything.

```
model3 <- gllvm(Y, X, lv.formula = ~(\text{Elevation.s} + \text{I}(\text{Elevation.s}^2)) * \text{Year.s},
               num.RR = 2, family = "ordinal", seed = 1)

# no newLV here: model3 has no unconstrained LVs, its axes follow from newX
SRpred3a <- predictSR(model3, newX = data.frame(\text{Elevation.s} = newELEV.s, \text{Year.s} = 0)) #
  ↪ 1978
SRpred3b <- predictSR(model3, newX = data.frame(\text{Elevation.s} = newELEV.s, \text{Year.s} = 6)) #
  ↪ 1984
```

Practical note: the trade-off is that all species respond to the same two linear combinations of the covariates; what differs between species is only how strongly, and with what sign, they load on those two axes. Also, ordination axes are identifiable only up to rotation and sign, so do not expect signs or orientations to match across separate fits.

```
# model3 has no unconstrained LVs to leak information, so this is already the fair
  ↪ comparison
goodnessOfFit(model3, measure = "AUC")
```

Compare this to the LV-zeroed `model2` number above, not the one conditional on the estimated latent variables: neither relies on a free per-site latent score, and `model3` additionally captures the year effect that `model2` cannot represent at all.

A naive richness benchmark

```
SRnaive <- rowSums(Y > 0)
SRnaiveMod <- glm(SRnaive ~ (\text{Elevation.s} + \text{I}(\text{Elevation.s}^2)) * \text{Year.s}, data = X, family =
  ↪ "poisson")

predSRnaive1 <- predict(SRnaiveMod, newdata = data.frame(\text{Elevation.s} = newELEV.s, \text{Year.s}
  ↪ = 0), se.fit = TRUE)
predSRnaive2 <- predict(SRnaiveMod, newdata = data.frame(\text{Elevation.s} = newELEV.s, \text{Year.s}
  ↪ = 6), se.fit = TRUE)
```

Summing to richness before modelling is where the covariance between species goes. A sum of presences is under-dispersed, its variance always below its mean. The Poisson pins it to the mean, $\text{Var}(SR_i) = E(SR_i)$, and it has no upper bound either, while true richness cannot exceed the size of the species pool, $SR_i \leq p$. So its uncertainty rests on an assumption the data need not obey, and near the edges of the range it can predict richness above the number of species available, which is not possible at all.

Side by side

```
par(mfrow = c(1, 2), mar = c(5,4,4,0))

plot(y = rowSums(Y > 0), X$Elevation, xlab = "Elevation", ylab = "Species richness",
     col = "lightgray", pch = 20 + X$Year.s - 1, main = "Richness (gllvm)")
abline(h = 10, v = 40, col = "gray", lty = "dotted", lwd = 2) # visual aid for comparison
```

```

lines(y = SRpred3a$expected$fit, x = newx, col = "red")
lines(y = SRpred3a$expected$lower, x = newx, col = "red", lty = "dashed")
lines(y = SRpred3a$expected$upper, x = newx, col = "red", lty = "dashed")
lines(y = SRpred3b$expected$fit, x = newx, col = "blue")
lines(y = SRpred3b$expected$lower, x = newx, col = "blue", lty = "dashed")
lines(y = SRpred3b$expected$upper, x = newx, col = "blue", lty = "dashed")

par(mar = c(5,0,4,0))
plot(y = rowSums(Y > 0), X$Elevation, xlab = "Elevation", ylab = NA,
     col = "lightgray", pch = 20 + X$Year.s - 1, main = "Richness (naive)", yaxt = "n")
abline(h = 10, v = 40, col = "gray", lty = "dotted", lwd = 2)

lines(y = exp(predSRnaive1$fit), x = newx, col = "red")
lines(y = exp(predSRnaive1$fit - 1.96*predSRnaive1$se.fit), x = newx, col = "red", lty =
  ↪ "dashed")
lines(y = exp(predSRnaive1$fit + 1.96*predSRnaive1$se.fit), x = newx, col = "red", lty =
  ↪ "dashed")
lines(y = exp(predSRnaive2$fit), x = newx, col = "blue")
lines(y = exp(predSRnaive2$fit - 1.96*predSRnaive2$se.fit), x = newx, col = "blue", lty =
  ↪ "dashed")
lines(y = exp(predSRnaive2$fit + 1.96*predSRnaive2$se.fit), x = newx, col = "blue", lty =
  ↪ "dashed")

```

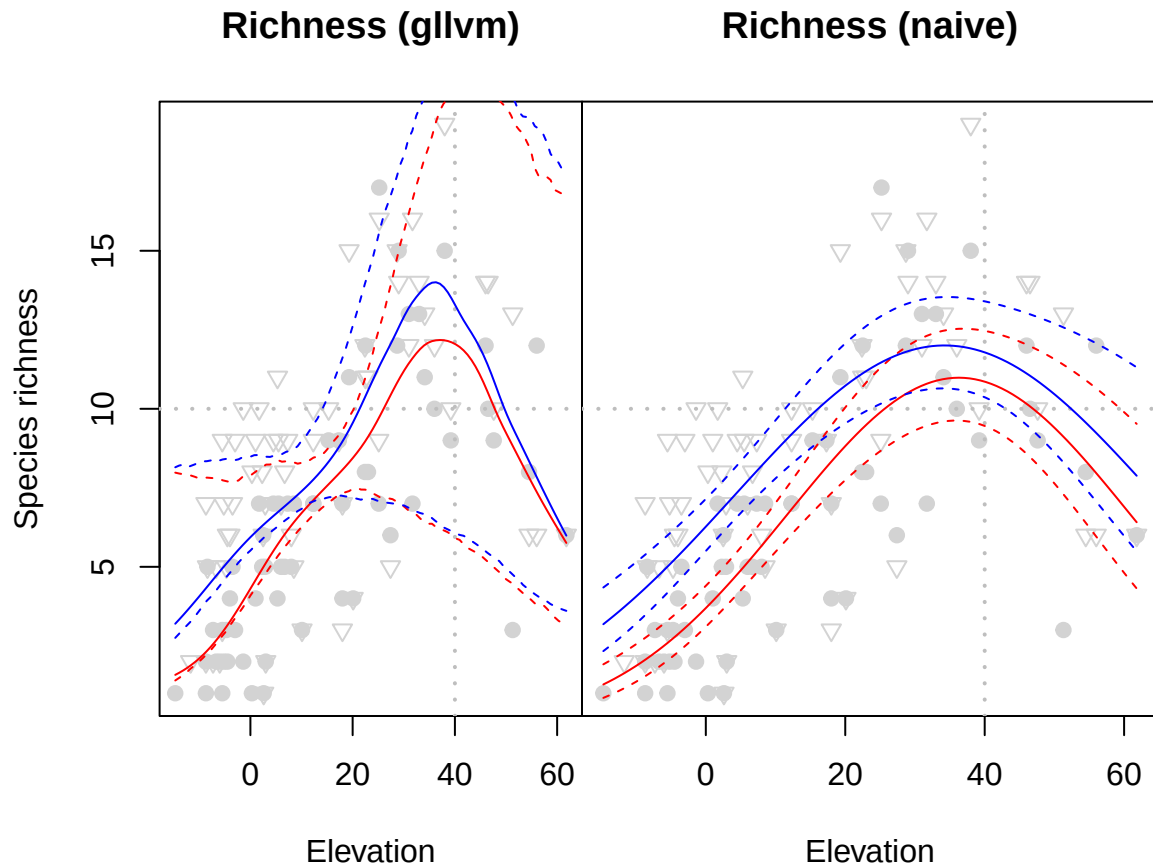


Figure 6: Predicted species richness by year (red = 1978, blue = 1984) from the constrained-ordination multispecies model (left) versus a naive Poisson GLM on total richness (right). Dashed lines are 95% confidence interval bounds for the mean.

```
par(mfrow = c(1, 1))
```

Same data, same axes: the bands of the naive Poisson GLM are narrower, especially where the two years' communities differ most. These are intervals for the mean, so their width mostly reflects how much is being estimated: six coefficients for a single scalar response, against a richness prediction that carries the uncertainty of every species' presence probability.

Beyond richness: composition

A better-behaved estimator of richness is the immediate gain from this route, but it is not the only one. Fitting richness directly gives you one number per site and stops there; to say anything about which species are involved you would have to run a separate ordination, and the two answers then come from two methods that need not agree with each other, with no shared uncertainty between them. Here both come out of the same fit. The species responses that produced the richness curves also define the axes below, so a change in predicted richness along elevation can be traced back to the species that caused it, and the shift in composition between 1978 and 1984 sits next to the richness predictions for those same years.

```
coefplot(model3)
```

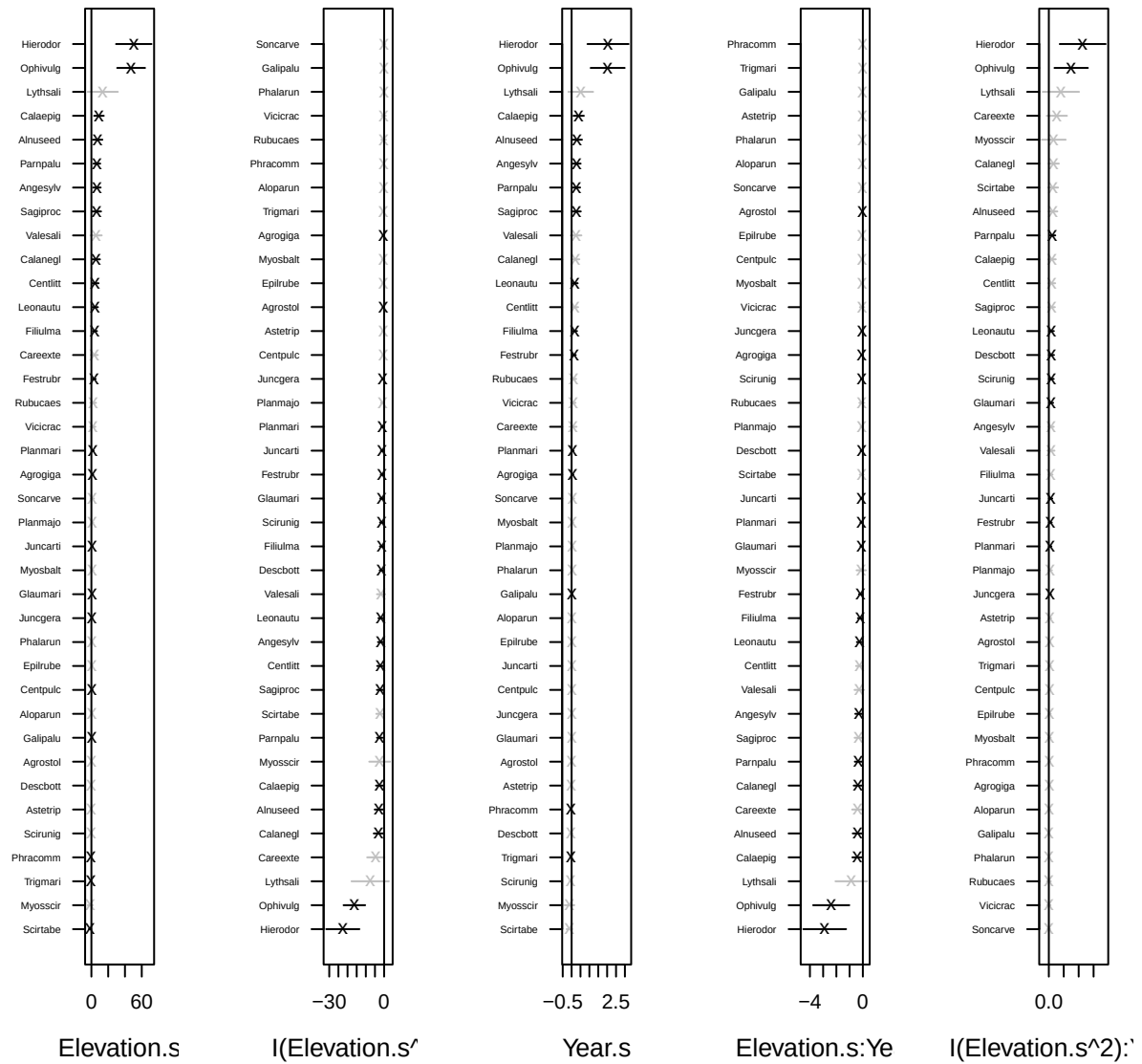


Figure 7: Species-specific effects of each term in the lv.formula of model3, reconstructed from the two constrained axes.

For a reduced-rank model, `coefplot` does not plot the canonical coefficients, it reconstructs the ordination-approximated species-specific effects, and shows them with standard errors.

```
par(mfrow = c(1, 2), mar = c(5,4,4,1))

ordiplot(model3, main = "Sites")

ordiplot(model3, biplot = TRUE, ind.spp = 15, s.colors = "transparent",
          main = "Species")
```

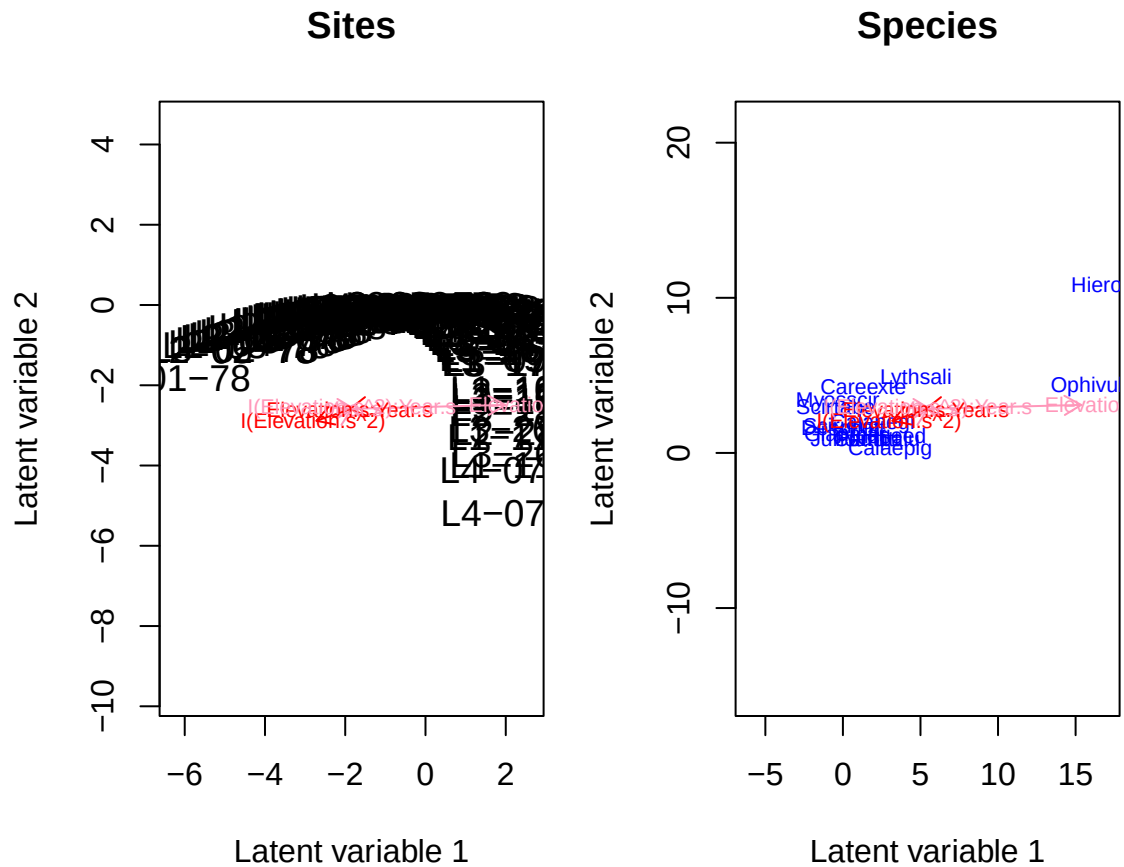


Figure 8: Constrained ordination from model3, split over two panels: site scores (left) and the 15 species with the longest loading vectors (right). Arrows are the environmental gradients.

```
par(mfrow = c(1, 1))
```

The two panels are the same ordination, drawn twice: the left keeps the site scores and drops the species, the right keeps the species and hides the sites (`s.colors = "transparent"`), so neither is buried under the other. `ind.spp = 15` limits the right panel to the 15 species with the longest loading vectors, since labelling all 38 is unreadable. Note that the biplot rescales species loadings against site scores, so the two panels share axes but not a numerical scale. The sites fall on a curve rather than scattering, which is what a fully constrained ordination gives you when its axes are built from elevation, its square, and year.

Richness tells you how many species, ordination tells you the composition. Two sites with the same richness can have very different composition, and we can use this dataset as an example.

```
P <- Y > 0
i <- which(rownames(X) == "L2-21-78")
j <- which(rownames(X) == "L4-07-78")

SRhat <- predictSR(model3, se.fit = FALSE)$expected$fit

c(observed_i = sum(P[i, ]), observed_j = sum(P[j, ]), shared = sum(P[i, ] & P[j, ]))
```

```
## observed_i observed_j      shared
##           7           6           0
```

```
round(c(predicted_i = SRhat[i], predicted_j = SRhat[j]), 2)
```

```
## predicted_i predicted_j
##           5.75           5.75
```

```
colnames(Y)[P[i, ]]
```

```
## [1] "Agrogiga" "Galipalu" "Glaumari" "Juncarti" "Phracomm" "Scirtabe" "Scirunig"
```

```
colnames(Y)[P[j, ]]
```

```
## [1] "Calaepig" "Festrubr" "Phalarun" "Rubucaes" "Soncarve" "Vicicrac"
```

model3 predicts an average of 5.75 species at both plots, and the two share not a single species. The first plot sits just 5.5 cm above the shoreline, and holds species that tolerate wet, brackish ground that is regularly under water: *Glaux maritima*, *Eleocharis uniglumis*, *Schoenoplectus tabernaemontani*, *Juncus articulatus* and *Phragmites australis*. The second sits at 61.8 cm, the top of the gradient, on ground that has been clear of the water long enough for tall grasses, meadow herbs and scrub to take hold: *Calamagrostis epigejos*, *Phalaris arundinacea*, *Festuca rubra*, *Vicia cracca*, *Sonchus arvensis* and *Rubus caesius*. Some of the latter prefer damp soil and others dry, but none of them will stand being flooded. Both ends of the gradient are species-poor, so a richness regression with the same covariates would predict much the same value at these two plots, and it would be right to. What it cannot add is that the two have not one species in common, and not because that is difficult to see: the two lists above show it plainly. It is that the information is not in the model. A richness regression is handed the totals and nothing else, so composition is not something it can be asked about, however obvious the answer is in the raw data. model3 is handed the species, so the fit that predicts richness is the same fit that knows which species produce it.

Both answers come out of that one fit, so we can put them in a single picture: the ordination, with each site coloured by the richness the model predicts for it.

```
pal <- hcl.colors(64, "Viridis")
cols <- pal[cut(SRhat, 64, labels = FALSE)]
pchs <- ifelse(X$Year == 1978, 16, 17) # circles 1978, triangles 1984

op <- ordiplot(model3, symbols = TRUE, s.colors = cols, pch = pchs,
               main = "Sites by predicted richness")

points(op$sites[c(i, j), ], pch = 1, cex = 2.5, lwd = 2)
text(op$sites[c(i, j), ], labels = rownames(X)[c(i, j)], pos = c(3, 2),
     offset = 0.9, cex = 0.7, font = 2)
legend("topleft", legend = c("1978", "1984"), pch = c(16, 17), bty = "n", cex = 0.7)

# colour gradient legend, drawn inside the plotting region
u <- par("usr"); dx <- diff(u[1:2]); dy <- diff(u[3:4])
bx <- u[1] + c(0.08, 0.12) * dx
by <- u[3] + c(0.08, 0.45) * dy

rasterImage(as.raster(rev(pal)), bx[1], by[1], bx[2], by[2], interpolate = TRUE)
rect(bx[1], by[1], bx[2], by[2])

at <- pretty(range(SRhat), 5)
at <- at[at >= min(SRhat) & at <= max(SRhat)]
yp <- by[1] + (at - min(SRhat)) / diff(range(SRhat)) * diff(by)
```

```
segments(bx[2], yp, bx[2] + 0.012 * dx, yp)
text(bx[2] + 0.018 * dx, yp, at, adj = 0, cex = 0.7)
text(bx[1], by[2] + 0.025 * dy, "Predicted SR", adj = 0, cex = 0.7)
```

Sites by predicted richness

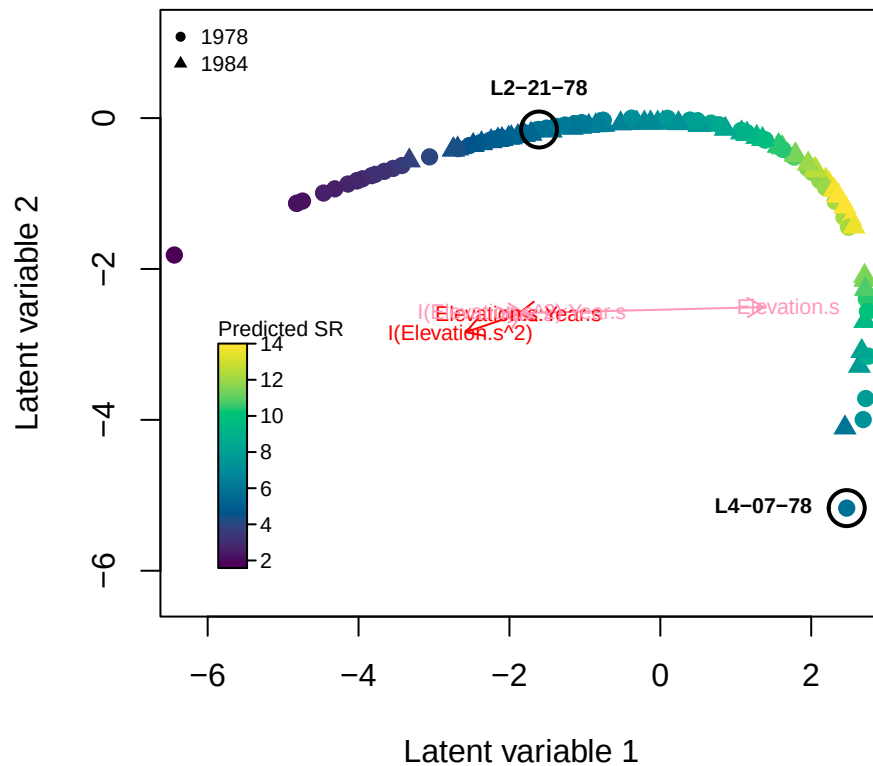


Figure 9: Site scores from model3, coloured by predicted species richness, with circles for 1978 and triangles for 1984. The two circled and labelled sites are the pair discussed above: the same predicted richness, far apart in the ordination.

The colouring is what makes the point. Richness is not a property of one end of the ordination: the same colour turns up in more than one place along the curve, and the two circled plots carry the same predicted value while sitting about as far apart as two equally rich sites can in this fit. The position tells you the composition, the colour tells you the richness, and both are read off one model.